
Greedy Best-First Search (GBFS)

Problem 1: Basic Graph Path (Heuristic to Goal)

You are given a **directed graph** where each node is connected to one or more other nodes. Your goal is to find a **path from a start node to a goal node** using **Greedy Best-First Search**.

- Each node has a **heuristic value ($h(n)$)** indicating an **estimate of the distance from that node to the goal**.
- Greedy BFS selects the next node with the **lowest heuristic value**, assuming it's closest to the goal.

Start Node: 'A'

Goal Node: 'G'

What GBFS Will Do:

1. Start at 'A', which has neighbors 'B' ($h=4$) and 'C' ($h=5$).
2. Choose 'B' because it has a lower heuristic.
3. 'B' has neighbors 'D' ($h=2$) and 'E' ($h=3$) → choose 'D'.
4. 'D' is a dead end → backtrack, explore 'E'.
5. 'E' has 'G', which is the goal.

```
graph = {  
  'A': ['B', 'C'],  
  'B': ['D', 'E'],  
  'C': ['F'],  
  'D': [],  
  'E': ['G'],  
  'F': ['G'],  
  'G': []  
}
```

```
heuristic = {  
  'A': 6,  
  'B': 4,  
  'C': 5,  
  'D': 2,  
  'E': 3,  
  'F': 2,  
  'G': 0  
}
```

Problem 2: Maze with Goal Heuristic

A robot is navigating a **2D maze/grid** represented by 0s (free path) and 1s (walls). The robot starts at the top-left corner and must reach the bottom-right corner using **Greedy Best-First Search**, guided by a **heuristic**, the Manhattan distance to the goal.

0 = Free cell (robot can move)

1 = Wall (robot cannot move)

Start position: (0, 0)

Goal position: (3, 3)

Greedy Best-First Search Strategy

- Only considers the **heuristic** (not the path cost so far)
- Always chooses the neighbor with the **lowest heuristic value**

```
maze = [  
  [0, 0, 0, 1, 0],  
  [1, 1, 0, 1, 0],  
  [0, 0, 0, 0, 0],  
  [0, 1, 1, 1, 0],  
  [0, 0, 0, 0, 0]  
]
```

Problem 3: City Navigation

```
graph = {  
  'S': ['A', 'B'],  
  'A': ['C'],  
  'B': ['D', 'E'],  
  'C': ['G'],  
  'D': [],  
  'E': ['G'],  
  'G': []  
}
```

```
heuristic = {  
  'S': 7, 'A': 6, 'B': 2, 'C': 1, 'D': 4, 'E': 2, 'G': 0  
}
```

Problem 4: Word Ladder (1-letter difference)

```
words = {
```

```

        'hit': ['hot'],
        'hot': ['dot', 'lot'],
        'dot': ['dog'],
        'lot': ['log'],
        'dog': ['cog'],
        'log': ['cog'],
        'cog': []
    }

```

```

    heuristic = {
        'hit': 3, 'hot': 2, 'dot': 2, 'dog': 1,
        'lot': 2, 'log': 1, 'cog': 0
    }

```

A* Search (A-Star)

$A^* = g(n) + h(n) \rightarrow$ combines cost-so-far and estimated future cost.

We are given a **graph** where each node represents a location, and edges represent paths between locations with associated **costs** (weights). The goal is to find the **shortest path** (minimum total cost) from a **start node** to a **goal node** using the **A* (A-Star) search algorithm**.

A* uses:

- $g(n) \rightarrow$ actual cost from the start to node n
- $h(n) \rightarrow$ estimated cost from node n to the goal (heuristic)
- $f(n) = g(n) + h(n) \rightarrow$ estimated total cost of the path through n

Find the lowest-cost path from **node 'A' to 'G'** using A*.

Why A* is Suitable

A* intelligently balances:

- **Actual cost so far** (g) — helps avoid paths that are long or expensive
- **Estimated cost to goal** (h) — guides toward goal quickly

A* avoids exhaustive search (like BFS/DFS) and avoids traps of using heuristic alone (as in Greedy).

Example Execution Summary

Start: 'A'

Goal: 'G'

It explores:

- 'A' $\rightarrow$ B $\rightarrow$ E $\rightarrow$ G'
- Total path cost = 1 (A-B) + 1 (B-E) + 2 (E-G) = 4

This is the **optimal path**, and A* correctly finds it.

Problem 1: Basic A* Graph Search

graph = {

```
'A': [('B', 1), ('C', 4)],  
'B': [('D', 3), ('E', 1)],  
'C': [('F', 5)],  
'D': [],  
'E': [('G', 2)],  
'F': [('G', 1)],  
'G': []  
}
```

```
heuristic = {  
'A': 7, 'B': 6, 'C': 4, 'D': 3, 'E': 2, 'F': 2, 'G': 0  
}
```

Problem 2: Robot in Grid with Cost and Heuristic

A robot is placed in the top-left corner of a 5x5 grid. Each cell contains a movement cost. The robot must reach the bottom-right corner of the grid, avoiding high-cost paths. Use A* search with the Manhattan distance as the heuristic.

```
grid = [  
[1, 3, 1, 2, 9],  
[7, 3, 4, 9, 2],  
[1, 7, 5, 5, 3],  
[2, 3, 2, 2, 1],  
[3, 1, 4, 2, 1]  
]
```

```
start = (0, 0)  
goal = (4, 4)
```
